Kilwater API - dokumentacja

Endpointy publiczne: `POST /api/zapytanie` (formularz zapytania o rejs), `POST /api/asystent` (asystent pytań w sekcji FAQ)
Pliki pomocnicze: `/sitemap.xml`, `/robots.txt`

```
https://kilwater.com
http://localhost:4520   (lokalnie, npm run dev)
```

Spis treści

1. Autoryzacja i limity
2. Konwencje i błędy
3. POST /api/zapytanie
4. Wartości pól wyboru i przykład
5. Odpowiedzi
6. Wysyłka z własnej strony
7. Dostarczanie zapytań
8. POST /api/asystent
9. Odpowiedzi asystenta
10. Jak asystent dobiera odpowiedź
11. GET /sitemap.xml i robots.txt
12. Dane osobowe
13. Lista kontrolna przed wdrożeniem
14. Rozwiązywanie problemów

Autoryzacja i limity

API nie wymaga kluczy, ciasteczek ani sesji. `POST /api/zapytanie` przyjmuje zapytania z formularza na stronie głównej i z dowolnego innego źródła, które wyśle poprawny JSON. Ochronę przed nadużyciami zapewniają limity i walidacja opisane niżej. Asystent (`/api/asystent`) ma własne, łagodniejsze limity opisane w jego sekcji.

Zabezpieczenie	Opis
Limit żądań	**5 żądań na 10 minut z jednego adresu IP**. Szóste i kolejne dostają `429`. Limit jest liczony w pamięci procesu serwera i zeruje się przy restarcie.
Rozmiar treści	Maksymalnie 10 000 bajtów. Większa treść dostaje `413`.
Typ treści	Wymagany nagłówek `Content-Type: application/json`. Inny typ dostaje `415`.
Pułapka na boty	Pole `website` musi być puste. Wypełnione oznacza bota: serwer odpowiada `200`, ale zapytania nie przetwarza.
Lista dozwolonych pól	Serwer przyjmuje tylko pola opisane w tej dokumentacji. Każde inne pole, np. `isAdmin` albo `role`, kończy się błędem `400` z nazwą tego pola.

Przykład żądania (curl)

```
curl -X POST https://kilwater.com/api/zapytanie \
  -H 'Content-Type: application/json' \
  -d '{"destination":"chorwacja","startDate":"2027-06-12","nights":"7",
       "guests":6,"cruiseType":"ze-skipperem","extras":["sup"],
       "name":"Anna Kowalska","email":"anna@example.com","phone":"",
       "message":"","consent":true,"website":""}'
```

Kolejność sprawdzania

1. Typ treści (`415`).
2. Rozmiar treści (`413`).
3. Limit żądań z adresu IP (`429`).
4. Poprawność JSON (`400` bez pola `errors`).
5. Pułapka na boty (`200` bez przetwarzania).
6. Walidacja pól (`400` z polem `errors`).
7. Wysyłka e-mail (`200` albo `503`).

Żądania odrzucone na krokach 1 i 2 nie zużywają limitu z kroku 3.

Konwencje i błędy

Zasada	Opis
Format	Odpowiedzi to JSON w UTF-8 z nagłówkiem `Cache-Control: no-store`.
Daty	Dzień bez godziny w formacie `YYYY-MM-DD`, np. `2027-06-12`. Porównanie z dniem dzisiejszym w UTC.
Wartości wyboru	Akwen, długość, rodzaj rejsu i dodatki przyjmują stałe identyfikatory (małe litery, myślniki). Pełna lista w sekcji „Wartości pól wyboru”.
Teksty	Spacje na początku i końcu pól tekstowych są usuwane przed walidacją.
Język	Komunikaty błędów są po polsku i mówią, co poprawić. Formularz pokazuje je bez zmian pod polami.

Odpowiedź zawsze ma pole `ok`. Pole `errors` pojawia się tylko przy błędach walidacji:

```
{"ok": false, "errors": {"email": "Podaj poprawny adres e-mail, np.
jan@przyklad.pl."}}
```

Kod	Kiedy
`200`	Zapytanie przyjęte i wysłane (albo zapisane w logu w trybie testowym). Asystent zwraca `200` przy każdej odpowiedzi, także „nie znam odpowiedzi”.
`204`	`GET /api/asystent`: serwer zaczął wczytywać model asystenta.
`400`	Treść nie jest poprawnym JSON-em (`{"ok": false}`) albo walidacja nie przeszła (`errors` z kluczem = nazwa pola).
`404`	Nieobsługiwana metoda, np. `GET /api/zapytanie`.
`413`	Treść żądania większa niż 10 000 bajtów.
`415`	Nagłówek `Content-Type` inny niż `application/json`.
`429`	Przekroczony limit z jednego IP: 5 zapytań albo 20 pytań do asystenta na 10 minut.
`503`	Wysyłka e-mail nie jest skonfigurowana na produkcji, dostawca poczty zwrócił błąd albo na serwerze brakuje modelu asystenta. Szczegóły tylko w logu serwera.

POST /api/zapytanie

POST `/api/zapytanie`

Przyjmuje zapytanie o rejs, sprawdza je i przekazuje brokerowi e-mailem z adresem klienta w polu Reply-To.

Treść żądania (JSON)

Parametr	Wymagany	Opis
`destination`	tak	Akwen. Jedna z wartości w sekcji „Wartości pól wyboru”.
`startDate`	tak	Data wypłynięcia `YYYY-MM-DD`, dziś albo później.
`nights`	tak	Długość rejsu jako tekst, np. `"7"`.
`guests`	tak	Liczba osób, liczba całkowita od 1 do 12. Przyjmuje też liczbę zapisaną jako tekst.
`cruiseType`	tak	Rodzaj rejsu.
`extras`	nie	Tablica dodatków, domyślnie pusta. Każdy dodatek najwyżej raz.

Parametr	Wymagany	Opis
`name`	tak	Imię i nazwisko, od 2 do 120 znaków.
`email`	tak	Adres e-mail, maksymalnie 200 znaków. Trafia do pola Reply-To wiadomości dla brokera.
`phone`	nie	Telefon, np. `+48 600 000 000` : cyfry, spacje, nawiasy i myślniki, co najmniej 7 znaków. Puste dozwolone.
`message`	nie	Wiadomość, maksymalnie 1000 znaków.
`consent`	tak	Musi mieć wartość `true` : akceptacja regulaminu i informacji z polityki prywatności.
`website`	nie	Pułapka na boty, zawsze puste. Patrz „Autoryzacja i limity”.

Wartości pól wyboru i przykład

Pola wyboru przyjmują tylko poniższe identyfikatory. Inna wartość kończy się błędem `400` z komunikatem dla tego pola.

`destination` **- akwen**

Wartość	Znaczenie
`chorwacja`	Chorwacja: Dalmacja i Kornati
`grecja`	Grecja: Cyklady i Wyspy Jońskie
`wlochy`	Włochy: Wybrzeże Amalfi, Sardynia, Sycylia
`mazury`	Mazury: Wielkie Jeziora Mazurskie
`nie-wiem`	Jeszcze nie wiem, akwen do ustalenia z brokerem

`nights` **- długość rejsu**

Wartość	Znaczenie
`3` , `5` , `7` , `10` , `14`	Liczba nocy. Formularz domyślnie zaznacza `7` .
`inna`	Inna długość, do ustalenia w rozmowie

`cruiseType` - rodzaj rejsu

Wartość	Znaczenie
`pelna-zaloga`	Z pełną załogą: skipper, hostessa i kucharz
`ze-skipperem`	Ze skipperem, bez reszty załogi
`bez-zalogi`	Bez załogi, na własne uprawnienia klienta

`extras` - dodatki

Wartość	Znaczenie
`sup`	Deski SUP
`snorkeling`	Sprzęt do snorkelingu
`ponton`	Ponton z silnikiem
`transfer`	Transfer z lotniska do mariny
`zaopatrzenie`	Zaopatrzenie jachtu przed przyjazdem
`posciel`	Pościel, ręczniki i sprzątanie po rejsie

Przykład żądania

Treść żądania `POST` z nagłówkiem `Content-Type: application/json`, ze wszystkimi polami:

```
{
  "destination": "chorwacja",
  "startDate": "2027-06-12",
  "nights": "7",
  "guests": 6,
  "cruiseType": "ze-skipperem",
  "extras": ["sup", "transfer"],
  "name": "Anna Kowalska",
  "email": "anna@example.com",
  "phone": "+48 600 000 000",
  "message": "Pierwszy rejs, dwoje dzieci.",
  "consent": true,
  "website": ""
}
```

Odpowiedzi

Odpowiedź - błąd walidacji (400)

```
{
  "ok": false,
  "errors": {
    "destination": "Wybierz akwen albo „Jeszcze nie wiem”.",
    "startDate": "Data wypłynięcia nie może być w przeszłości.",
    "guests": "Maksymalnie 12 osób na jednym jachcie.",
    "name": "Podaj imię i nazwisko.",
    "email": "Podaj poprawny adres e-mail, np. jan@przyklad.pl.",
    "phone": "Podaj numer w formacie +48 600 000 000 albo zostaw puste.",
    "consent": "Zaznacz akceptację regulaminu, żebyśmy mogli przygotować ofertę."
  }
}
```

Odpowiedź - niedozwolone pola (400)

```
{
  "ok": false,
  "errors": {
    "isAdmin": "To pole nie jest obsługiwane.",
    "role": "To pole nie jest obsługiwane."
  }
}
```

Odpowiedź - pozostałe błędy (400, 413, 415, 429, 503)

```
{"ok": false}
```

Odpowiedź - sukces (200)

```
{"ok": true}
```

Znaczenie kluczy w odpowiedzi

Klucz	Opis
`ok`	`true`, gdy zapytanie zostało przyjęte. `false` przy każdym błędzie.
`errors`	Tylko przy błędzie walidacji. Obiekt: nazwa pola → komunikat po polsku z informacją, co poprawić. Pola, których serwer nie obsługuje, dostają komunikat „To pole nie jest obsługiwane.”.

Sukces nie zwraca identyfikatora zapytania, bo serwer niczego nie zapisuje w bazie. Zapytanie istnieje tylko jako e-mail u brokera.

Wysyłka z własnej strony

Formularz na kilwater.com robi dokładnie to, co poniższy kod. Tak samo można wysłać zapytanie z innej strony albo aplikacji, np. z kampanii albo strony partnera.

```
const response = await fetch("https://kilwater.com/api/zapytanie", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(inquiry),
});
const result = await response.json().catch(() => ({}));

if (response.ok) showThankYou();
else if (response.status === 400 && result.errors) showFieldErrors(result.errors);
else if (response.status === 429) showMessage("Odczekaj kilka minut i spróbuj
ponownie.");
else showMessage("Nie udało się wysłać zapytania. Spróbuj ponownie za chwilę.");
```

Sytuacja	Co pokazać użytkownikowi
`200`	Potwierdzenie i informację, że broker odezwie się na podany adres e-mail.
`400` z `errors`	Komunikat pod każdym polem z obiektu `errors` i fokus na pierwszym błędnym polu.
`429`	Prośbę o odczekanie kilku minut. Ponowne wysłanie od razu znowu dostanie `429`.
Inny błąd albo brak sieci	Ogólny komunikat z prośbą o ponowienie i kontaktem e-mail `kontakt@kilwater.com`.

Dostarczanie zapytań

Po walidacji serwer wysyła zapytanie przez API Resend jako zwykły tekst (bez HTML). Temat ma postać `Zapytanie o rejs: Chorwacja, 2027-06-12`, a pole Reply-To wskazuje adres klienta, więc broker odpowiada bezpośrednio z poczty.

Zmienna	Opis
`RESEND_API_KEY`	Klucz API Resend. Tylko na serwerze, nigdy w przeglądarce.
`INQUIRY_TO_EMAIL`	Adres, na który przychodzą zapytania, np. `kontakt@kilwater.com`.
`INQUIRY_FROM_EMAIL`	Nadawca zweryfikowany w Resend.
`INQUIRY_DRY_RUN`	`1` = zapytanie tylko w logu, do testów.

Treść e-maila (tryb testowy wypisuje to samo w logu)

```
Akwen: Chorwacja
Data wypłynięcia: 2027-06-12
Długość: 7 nocy
Liczba osób: 6
Rodzaj rejsu: Ze skipperem
Dodatki: Deski SUP, Transfer z lotniska

Imię i nazwisko: Anna Kowalska
E-mail: anna@example.com
Telefon: +48 600 000 000

Wiadomość: Pierwszy rejs, dwoje dzieci.
```

Zachowanie zależnie od konfiguracji

Konfiguracja	Zachowanie
Wszystkie trzy zmienne Resend ustawione	Wysyłka przez Resend. Błąd dostawcy → `503`.
Brak zmiennych, `npm run dev`	Zapytanie w logu, odpowiedź `200`.
Brak zmiennych, `INQUIRY_DRY_RUN=1`	Zapytanie w logu, odpowiedź `200` (używane przez testy e2e).
Brak zmiennych na produkcji	Odpowiedź `503`, żeby zapytania nie ginęły po cichu.

POST /api/asystent

POST `/api/asystent`

Odpowiada na pytanie o rejs gotowym tekstem z bazy wiedzy Kilwater. Działa w całości na serwerze strony, przez lokalny model, bez zewnętrznego API i bez opłat za pytania. Gdy w bazie nie ma odpowiedzi, kieruje na `kontakt@kilwater.com`.

GET `/api/asystent`

Bez parametrów. Zaczyna wczytywać model w tle i od razu zwraca `204` bez treści. Strona wysyła to żądanie, gdy sekcja FAQ zbliża się do ekranu, żeby pierwsze pytanie nie czekało na model.

Treść żądania (JSON)

Parametr	Wymagany	Opis
`question`	tak	Pytanie, od 3 do 300 znaków. Serwer zamienia ciągi spacji i nowe linie na jedną spację i obcina spacje na brzegach.
`website`	nie	Pułapka na boty, zawsze puste. Wypełnione daje odpowiedź `unknown` bez przetwarzania pytania.

Przykład żądania (curl)

```
curl -X POST https://kilwater.com/api/asystent \
  -H 'Content-Type: application/json' \
  -d '{"question":"czy na jacht wejdzie mój labrador","website":""}'
```

Limity i kolejność sprawdzania

Krok	Odpowiedź przy błędzie
1. Nagłówek `Content-Type: application/json`	`415` i `{"ok": false}`
2. Treść do 2000 bajtów	`413` i `{"ok": false}`
3. Limit 20 pytań na 10 minut z jednego IP (zmienna `ASSISTANT_RATE_LIMIT`)	`429` i `"reason": "rate_limited"`
4. Poprawny JSON, tylko pola `question` i `website`	`400` z polem `error`

Krok	Odpowiedź przy błędzie
5. Filtr danych kontaktowych i linków	`200` z `"kind": "blocked"`, pytanie nie trafia do modelu
6. Model dostępny na serwerze	`503` i `"reason": "unavailable"`

Limit asystenta jest wyższy niż formularza, bo pytanie nic nie kosztuje. Chroni tylko procesor serwera przed zalaniem żądaniami.

Odpowiedzi asystenta

Odpowiedź - znaleziona w bazie (200, `answer`)

```
{
  "ok": true,
  "kind": "answer",
  "text": "Najlepszy czas w Grecji to czerwiec i wrzesień, a szczyt sezonu przypada
           na lipiec i sierpień. W lipcu i sierpniu na Cykladach wieje silne
meltemi…",
  "link": { "href": "/#kiedy-plynac", "label": "Zobacz kalendarz sezonu" },
  "matchedQuestion": "Kiedy wieje meltemi?",
  "related": []
}
```

Odpowiedź - brak pewności (200, `unsure`)

```
{
  "ok": true,
  "kind": "unsure",
  "text": "Nie mam pewnej odpowiedzi. Sprawdź podobne pytania poniżej albo napisz na
kontakt@kilwater.com.",
  "related": ["Co to jest rejs ze skipperem?", "Jakie jachty macie?", "Kto jest w
załodze jachtu?"]
}
```

Odpowiedź - poza bazą (200, `unknown`) i filtr (200, `blocked`)

```
{"ok": true, "kind": "unknown", "related": [],
 "text": "Nie znam odpowiedzi na to pytanie. Napisz do nas na kontakt@kilwater.com,
a broker odpowie osobiście."}

{"ok": true, "kind": "blocked", "related": [],
 "text": "Nie wpisuj tu danych kontaktowych. Jeśli broker ma się do Ciebie odezwać,
wyślij formularz zapytania niżej na stronie."}
```

Odpowiedź - błędy (400, 429, 503)

```
{"ok": false, "error": "Wpisz pytanie, co najmniej 3 znaki."}
{"ok": false, "error": "To pole nie jest obsługiwane."}
{"ok": false, "reason": "rate_limited"}
{"ok": false, "reason": "unavailable"}
```

Znaczenie kluczy w odpowiedzi

Klucz	Opis
`ok`	`true` przy każdej odpowiedzi asystenta, także `unknown` i `blocked`. `false` tylko przy błędzie żądania.
`kind`	Rodzaj odpowiedzi, patrz tabela niżej.
`text`	Tekst do pokazania użytkownikowi. Zawsze gotowy tekst z bazy albo stały komunikat, nigdy zdanie wygenerowane przez model.
`link`	Opcjonalny. `href` i `label` przycisku pod odpowiedzią, np. do formularza albo kalendarza sezonu.
`matchedQuestion`	Tylko przy `answer`. Pytanie z bazy, do którego dopasowano pytanie użytkownika. Strona pokazuje je, gdy różni się od wpisanego.
`related`	Lista pytań z bazy do kliknięcia. Przy `answer` inne mocno pasujące tematy, przy `unsure` najbliższe pytania. Kliknięcie wysyła to pytanie jeszcze raz.
`error`	Tylko przy `400`. Komunikat po polsku, co poprawić.
`reason`	Tylko przy `429` i `503`: `rate_limited` albo `unavailable`.

Wartości `kind`

Wartość	Kiedy	Co pokazuje strona
`answer`	Pytanie wyraźnie pasuje do pytania z bazy.	Tekst, ewentualny przycisk z `link` i podobne pytania.
`unsure`	Kilka pytań z bazy pasuje podobnie słabo.	Komunikat, przycisk e-mail i pytania z `related`.
`unknown`	Nic w bazie nie pasuje, np. pytanie spoza tematu.	Komunikat i przycisk e-mail z wpisanym pytaniem.
`blocked`	W pytaniu jest e-mail, numer telefonu albo link.	Komunikat z prośbą o usunięcie danych albo opisanie pytania słowami.

Jak asystent dobiera odpowiedź

Asystent nie pisze własnych zdań. Lokalny model zamienia pytanie na wektor znaczenia i porównuje go z każdym sformułowaniem w bazie, a serwer pokazuje odpowiedź przypisaną do najbliższego. Dlatego nie da się go użyć do tematów spoza rejsów.

Element	Opis
Model	`Xenova/bge-m3` (licencja MIT, ok. 560 MB) przez `@huggingface/transformers` i ONNX Runtime. Działa na procesorze, bez karty graficznej.
Baza wiedzy	`src/data/assistant-answers.ts`: 40 odpowiedzi i 233 sformułowania pytań. FAQ ze strony z typowymi sformułowaniami (`alsoAsked` w `faq.ts`) oraz odpowiedzi składane z danych strony: akweny, sezon, trasy, rodzaje rejsów, załoga, dodatki, kontakt.
Pewna odpowiedź	Podobieństwo co najmniej 0,80 albo co najmniej 0,70 z przewagą 0,04 nad drugą odpowiedzią.
Brak pewności	Inaczej, ale są pytania z podobieństwem od 0,64. Pokazuje do trzech takich pytań.
Brak odpowiedzi	Wszystkie podobieństwa poniżej 0,64. W pomiarze pytania spoza tematu dostawały najwyżej 0,62.
Wydajność	Wczytanie modelu po starcie serwera trwa kilka sekund, kolejne odpowiedzi kilkadziesiąt milisekund. Proces z modelem zajmuje ok. 1 GB RAM.

Wdrożenie modelu

```
npm install
npm run assistant:model   # pobiera model do ./models (raz, ok. 560 MB)
npm run build
HOST=0.0.0.0 PORT=8080 node ./dist/server/entry.mjs   # start z katalogu projektu
```

Folder `models/` jest w `.gitignore`. W działaniu serwer nie pobiera niczego z internetu (`allowRemoteModels = false`), więc bez kroku `assistant:model` asystent odpowiada `503`.

Jak nauczyć asystenta nowego pytania

1. Gdy pytanie pasuje do istniejącej odpowiedzi, dopisz nowe sformułowanie do `alsoAsked` w `faq.ts` albo do `questions` w `assistant-answers.ts`.
2. Gdy to nowy temat, dodaj wpis z `id`, kilkoma sformułowaniami, odpowiedzią opartą na danych firmy i opcjonalnym `link`. Nie wpisuj cen ani obietnic, których nie potwierdził właściciel.
3. Uruchom `npm run assistant:eval`. Zestaw 48 pytań kontrolnych sprawdza parafrazy, pytania bliskie tematu bez odpowiedzi, pytania spoza tematu i filtr danych kontaktowych.

GET /sitemap.xml i robots.txt

GET `/sitemap.xml`

Mapa strony dla wyszukiwarek, generowana przy budowaniu, bez parametrów. Adresy powstają z `site` w `astro.config.mjs`, a strony 404 w mapie nie ma.

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url><loc>https://kilwater.com/</loc></url>
  <url><loc>https://kilwater.com/polityka-prywatnosci</loc></url>
  <url><loc>https://kilwater.com/regulamin</loc></url>
</urlset>
```

Plik `/robots.txt` pozwala indeksować całą stronę poza `/api/` i wskazuje tę mapę:

```
User-agent: *
Allow: /
Disallow: /api/

Sitemap: https://kilwater.com/sitemap.xml
```

Dane osobowe

Gdzie trafiają dane klienta. Podstawy prawne opisuje `/polityka-prywatnosci`.

Dane	Gdzie trafiają	Jak długo
Wszystkie pola formularza	E-mail do brokera, wysłany przez Resend jako pośrednika. Serwer strony niczego nie zapisuje w bazie ani na dysku.	Zgodnie z polityką prywatności.
Adres IP	Pamięć procesu serwera, tylko do liczenia limitu żądań.	Do 10 minut, znika też przy restarcie serwera.
Treść pytania do asystenta	Tylko pamięć procesu serwera na czas dopasowania odpowiedzi. Nie trafia do firm zewnętrznych, a log zapisuje wyłącznie rodzaj odpowiedzi.	Nie jest przechowywana.
Pełna treść zapytania w logu	Log serwera, tylko w `npm run dev` i przy `INQUIRY_DRY_RUN=1`.	Do wyczyszczenia logu.
Dane w przeglądarce	Formularz nie zapisuje ciasteczek ani `localStorage`.	Brak przechowywania.

Zasady dla integracji

- Wysyłaj zapytanie tylko z `consent: true` zaznaczonym przez klienta, nigdy automatycznie.
- Nie dopisuj danych, których klient sam nie podał, np. z systemu CRM.
- Informacje o zdrowiu, np. alergie, przekazuj tylko wtedy, gdy klient sam je wpisał.
- Nie zapisuj w logach własnej aplikacji treści zapytań ani pytań do asystenta.

Prośby klientów o ich dane

Serwer strony nie przechowuje zapytań, więc prośby obsługuje się w skrzynce brokera, w ciągu miesiąca (art. 12 ust. 3 RODO):

Prośba	Co zrobić
Dostęp do danych	Wyszukaj w skrzynce wiadomości z adresem klienta w Reply-To i prześlij mu ich treść.
Usunięcie danych	Usuń te wiadomości ze skrzynki i z kosza, a potem potwierdź usunięcie klientowi.
Sprostowanie	Odpowiedz na wiadomość z poprawionymi danymi, żeby historia zawierała aktualną wersję.

Lista kontrolna przed wdrożeniem

Endpoint działa od razu po buildzie, ale na produkcji bez tych kroków zapytania nie dotrą do brokera albo będą słabiej chronione.

Krok	Jak sprawdzić
Domena zweryfikowana w Resend	Panel Resend pokazuje status „Verified" przy `kilwater.com`.
Zmienne Resend ustawione na serwerze	Testowe zapytanie z formularza przychodzi na `INQUIRY_TO_EMAIL`, a nie kończy się `503`.
`INQUIRY_DRY_RUN` wyłączone	Zmienna nie istnieje albo jest pusta w środowisku produkcyjnym.
Prawdziwy adres IP klienta	Za reverse proxy serwer musi widzieć adres klienta, inaczej wszyscy dzielą jeden limit 5 żądań.
Nagłówki bezpieczeństwa	Hosting ustawia CSP, HSTS, `Referrer-Policy`, `Permissions-Policy` i `frame-ancestors`.
Ochrona przed botami	Przy większym ruchu dodaj Cloudflare Turnstile do formularza i sprawdzanie tokenu w endpoincie.
Model asystenta na serwerze	Po `npm run assistant:model` pytanie w sekcji FAQ dostaje odpowiedź, a nie komunikat o niedostępności. Serwer ma co najmniej 2 GB RAM.
Pytania kontrolne asystenta	`npm run assistant:eval` przechodzi w całości po każdej zmianie bazy odpowiedzi, przed wdrożeniem.
Rozgrzany asystent po starcie	Skrypt startowy albo healthcheck wysyła `GET /api/asystent` i dostaje `204`, więc pierwszy klient nie czeka na wczytanie modelu.
Adres strony	`site` w `astro.config.mjs` wskazuje docelową domenę, a `/sitemap.xml` zwraca właściwe adresy.

Rozwiązywanie problemów

Najczęstsze objawy po wdrożeniu i ich przyczyny. Szczegóły błędów wysyłki są zawsze w logu serwera, nigdy w odpowiedzi API.

Objaw	Przyczyna i rozwiązanie
Każde zapytanie kończy się `503`	Na serwerze brakuje jednej ze zmiennych Resend albo klucz jest nieważny. Log pokazuje `brak konfiguracji wysyłki e-mail` albo `błąd wysyłki e-mail` z kodem od Resend.
Klienci szybko dostają `429`	Serwer stoi za reverse proxy i widzi jeden adres IP dla wszystkich. Przekaż adres klienta z proxy do serwera, żeby limit liczył się osobno dla każdego.
Maile z zapytaniami trafiają do spamu	Domena nadawcy nie ma rekordów SPF i DKIM z panelu Resend. Dodaj je w DNS i poczekaj na status „Verified”.
Data wypłynięcia „w przeszłości” dla dzisiejszej daty	Serwer porównuje daty w UTC, więc tuż po północy czasu polskiego dzisiejsza data bywa dla niego jeszcze jutrem.
Asystent zawsze odpowiada „chwilowo niedostępny”	Brak modelu w `./models` albo serwer uruchomiony z innego katalogu. Log pokazuje `model nie jest dostępny`. Uruchom `npm run assistant:model` i startuj serwer z katalogu projektu.
Pierwsza odpowiedź asystenta trwa kilka sekund	Po starcie serwera model wczytuje się do pamięci. Strona robi to z wyprzedzeniem przez `GET /api/asystent`, a po wdrożeniu można wysłać to żądanie od razu, np. w skrypcie startowym.
Serwer restartuje się po pierwszym pytaniu do asystenta	Za mało pamięci: proces z modelem potrzebuje ok. 1 GB RAM. Wybierz serwer z co najmniej 2 GB albo zwiększ limit pamięci kontenera.
Asystent daje odpowiedź na inny temat	Pytanie jest blisko tematu, a w bazie brakuje pasującego sformułowania. Dopisz je do bazy i uruchom `npm run assistant:eval`.
Formularz pokazuje „Brak połączenia z internetem”	Żądanie nie dotarło do serwera, np. przez blokadę sieci albo wyłączony serwer.